Nigeria Security App

System Architecture, Production Roadmap & Tooling

Integrated mobile-first safety and field-operations platform for Nigerian public-safety agencies

Contents: System Architecture (with diagrams) · Working-Backwards Roadmap · Tooling

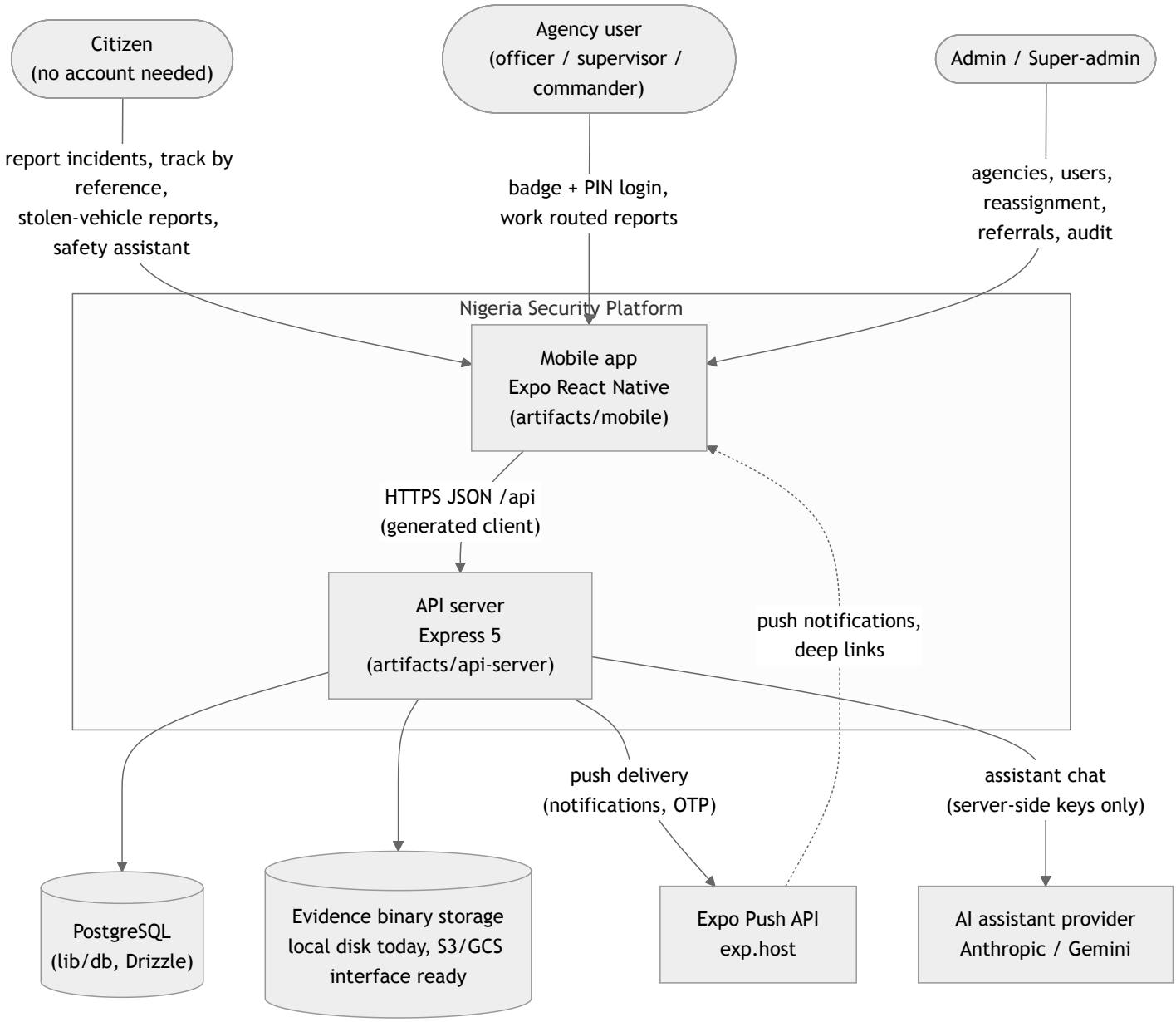
Generated 10 July 2026

Nigeria Security App — System Architecture

> Audience: engineers and reviewers who need to understand how the system is built, where the trust boundaries are, and what must change before it serves millions of users. Companion documents: ROADMAP.md (the working-backwards plan to production) and TOOLING.md (developer and operations tooling).
>
> Diagrams are Mermaid — they render on GitHub and in VS Code (with the Mermaid extension or built-in Markdown preview).

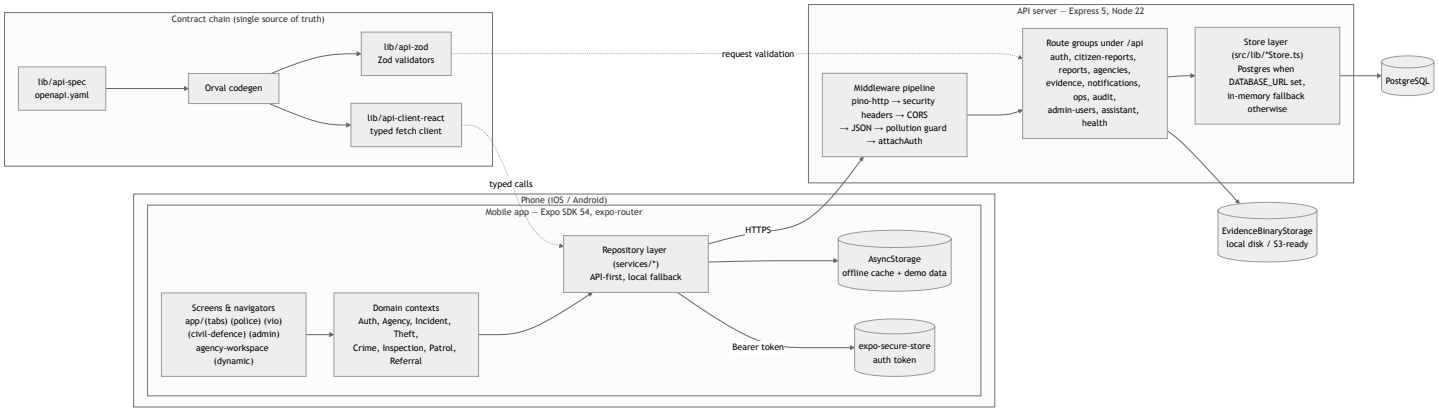
1. System context

The platform connects three kinds of people — citizens, agency personnel (FRSC, Police, VIO, NSCDC, plus admin-created agencies), and platform administrators — through one mobile app and one API.



Key property: **citizens are anonymous**. Public endpoints (report submission, tracking by reference, evidence attach) require no account — which makes them the primary abuse surface and the reason rate limiting, validation, and idempotency live there.

2. Container view



The contract chain

`lib/api-spec/openapi.yaml` is the single source of truth. `pnpm --filter @workspace/api-spec codegen` runs Orval (lib/api-spec/orval.config.ts) and generates both sides of the wire:

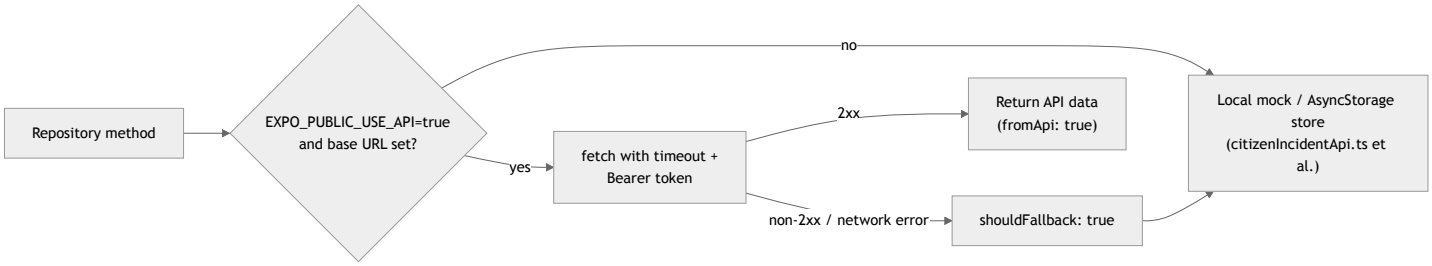
- **lib/api-zod** — runtime Zod validators the server uses on every request body.
- **lib/api-client-react** — the typed client the mobile app calls, with a hand-written mutator (custom-fetch.ts) that attaches the bearer token and normalizes errors.

Server and client therefore cannot drift from each other without the spec changing — and CI should enforce that codegen output is never stale (see TOOLING.md §6).

3. Mobile app architecture

Data layer: API-first with local fallback

Every repository in artifacts/mobile/services/ follows one pattern, implemented by `mobileApiFetch` in apiClient.ts:



Callers never throw on API failure — the app degrades to its offline cache instead of breaking in the field. Repositories: report, auth, user, agency, notification, audit, referral, duty, evidence, assistant (all in `services/`).

Navigation and role routing

expo-router file-based navigation. app/index.tsx is the public landing; on login, `routeForUser` in AuthContext.tsx routes by agency/role:

User	Workspace
FRSC	`app/(tabs)` — home, cases, map, alerts, profile
Police	`app/(police)` — crime reports, vehicle check, stolen alerts
VIO	`app/(vio)` — inspections, certificates
NSCDC	`app/(civil-defence)` — incidents, alerts
Admin / super-admin	`app/(admin)` — agencies, users, referrals, audit (hidden tabs)
Any admin-created agency (DSS, Fire, custom)	`app/agency-workspace.tsx` — generic scalable workspace

Every group `_layout.tsx` re-checks `user.agency`/role and redirects to `/` or `/unauthorized` — client-side defense in depth on top of server RBAC.

Offline sync and idempotency

Officer incidents queue in IncidentContext.tsx with `pendingSync: true` when offline (connectivity polled via expo-network every 15s). `syncPending()` retries each queued incident up to 2 attempts. Duplicates are impossible server-side because every submission carries a **stable `clientId`** (the incident's own id), and `citizen_reports.client_id` has a unique index — a retry after a lost response returns the original report and reference instead of creating a second row.

Token handling

Tokens live in **expo-secure-store** (keychain/keystore) via authToken.ts, with AsyncStorage fallback only where SecureStore is unavailable (web). On startup the app validates the session (`GET /auth/me`) and proactively rotates the token (`POST /auth/refresh`); it clears the token only on explicit 401/403, never on network errors.

4. API server architecture

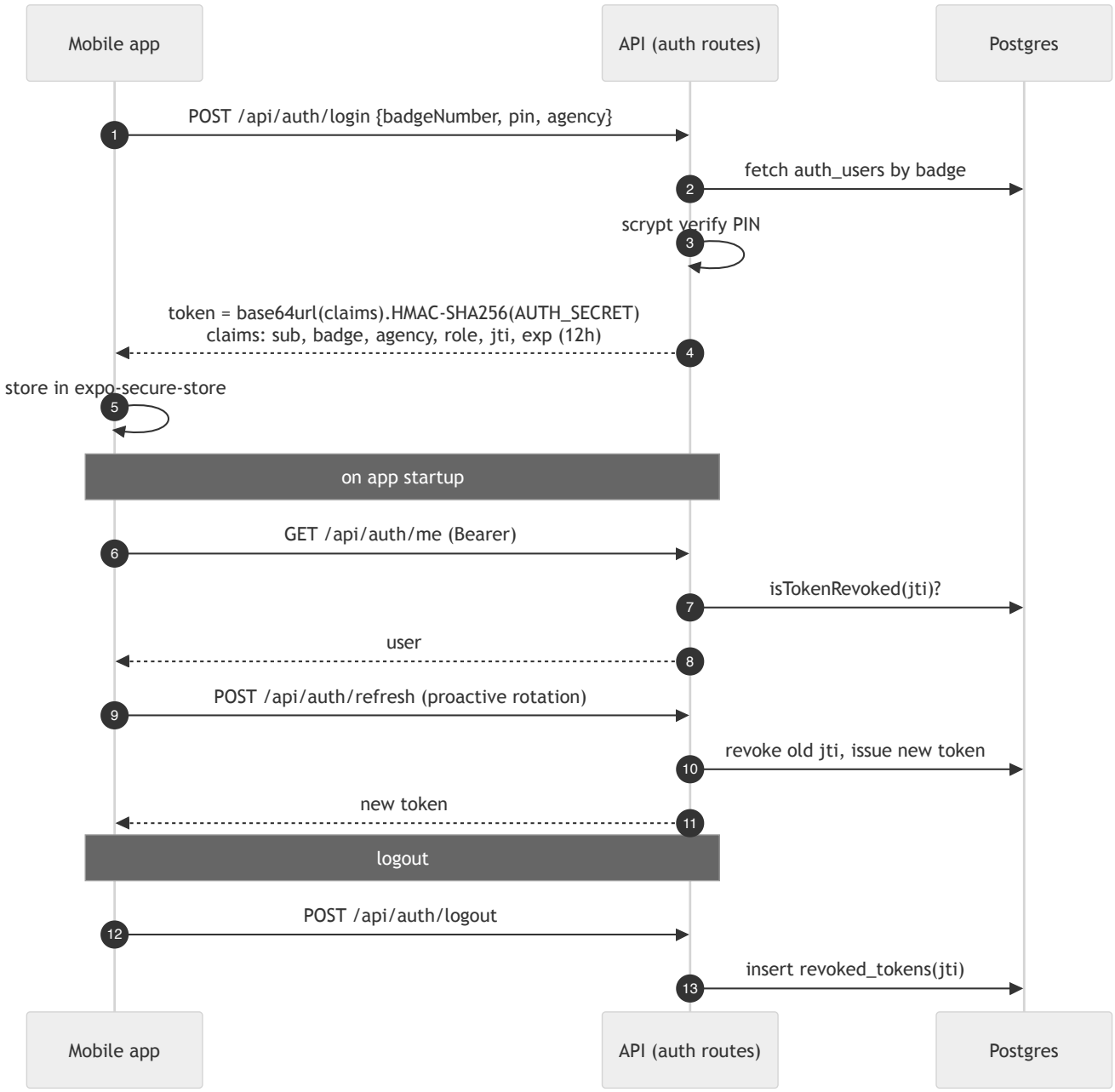
Middleware pipeline (order matters)

Defined in app.ts:

1. **pino-http** — structured logs; honors inbound `X-Request-Id` (≤128 chars) or generates a UUID; echoes it on the response for cross-system tracing.
2. **securityHeaders** — `X-Content-Type-Options` , `X-Frame-Options: DENY` , strict CSP (`default-src 'none'`), Referrer-Policy, CORP; HSTS in production.
3. **CORS** (currently open — see §8).
4. **Body parsing** + **prototype-pollution guard** (recursive rejection of `__proto__` / `constructor` / `prototype` keys → 400).
5. **attachAuth** — parses/verifies the bearer token, checks revocation; non-blocking so public routes work, and route guards (`requireAuth` / `requireAdmin` / `requireAgencyAccess`) enforce access per route.
6. Routers under `/api` , then a consistent `{error, code}` JSON error surface.

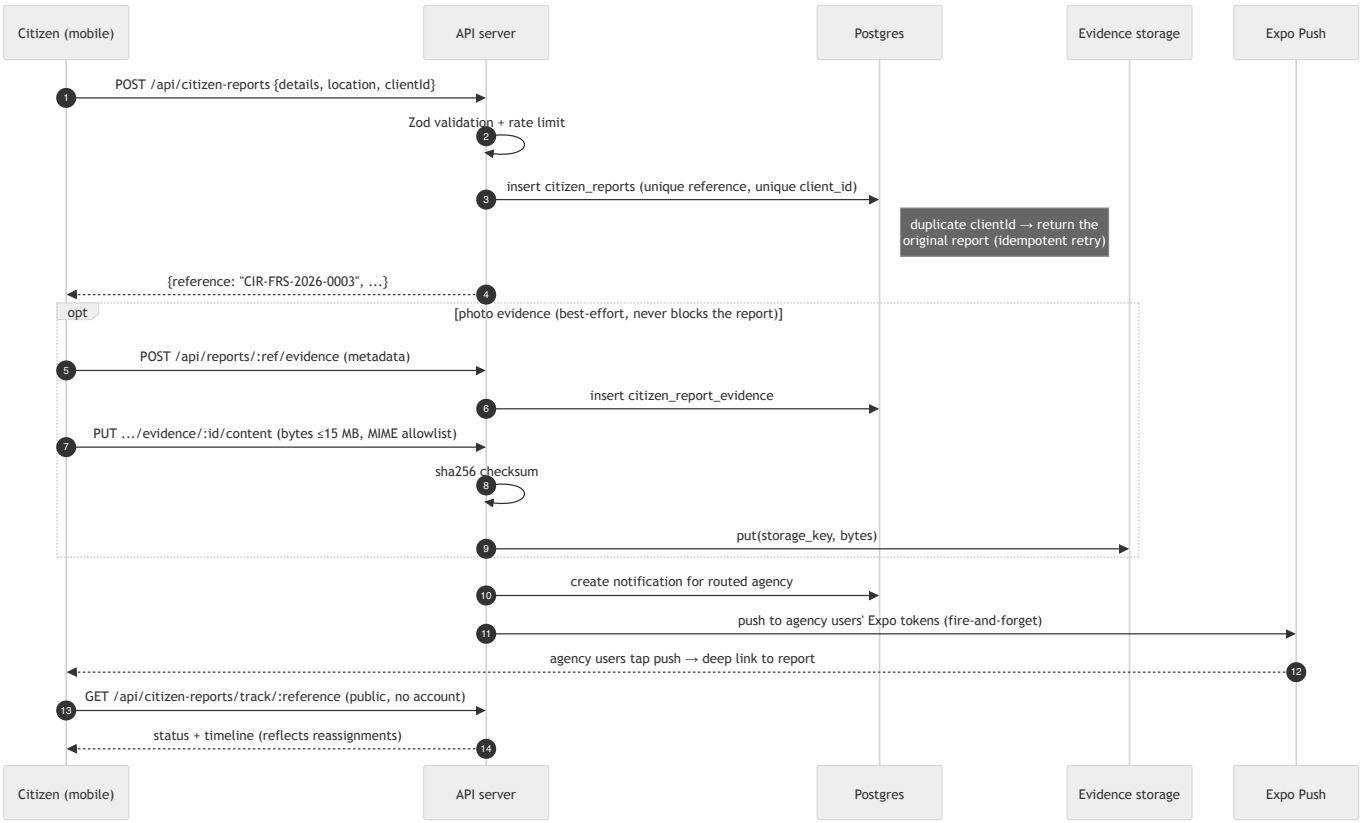
Rate limiting is per-route (fixed window, in-process): login/PIN-reset 20 per 15 min, OTP 5 per 15 min per IP+badge, citizen submissions and evidence uploads capped. **In-process means per-instance** — see §9 for the multi-instance implication.

Authentication and authorization



- Tokens are stateless HMAC (no session store); revocation is DB-backed by `jti` in the `revoked_tokens` table (in-memory Map without a database).
- `AUTH_SECRET` signs both auth tokens **and** evidence download URLs; production refuses to start without it (≥16 chars).
- RBAC roles: `citizen | officer | supervisor | commander | admin | super_admin`. Guards: `requireAuth` (401), `requireAdmin` (403), `requireAgencyAccess` (admin bypass, else agency match). The mobile capability matrix (lib/permissions.ts) is a UI hint only — the server always re-enforces.
- Self-service PIN reset: OTP (6-digit, sha256-hashed, 10-min TTL, 5 attempts) is delivered by **push to the user's own registered devices** — never to a caller-supplied address — then a one-shot reset grant allows the PIN change.

The citizen report path (the core flow)



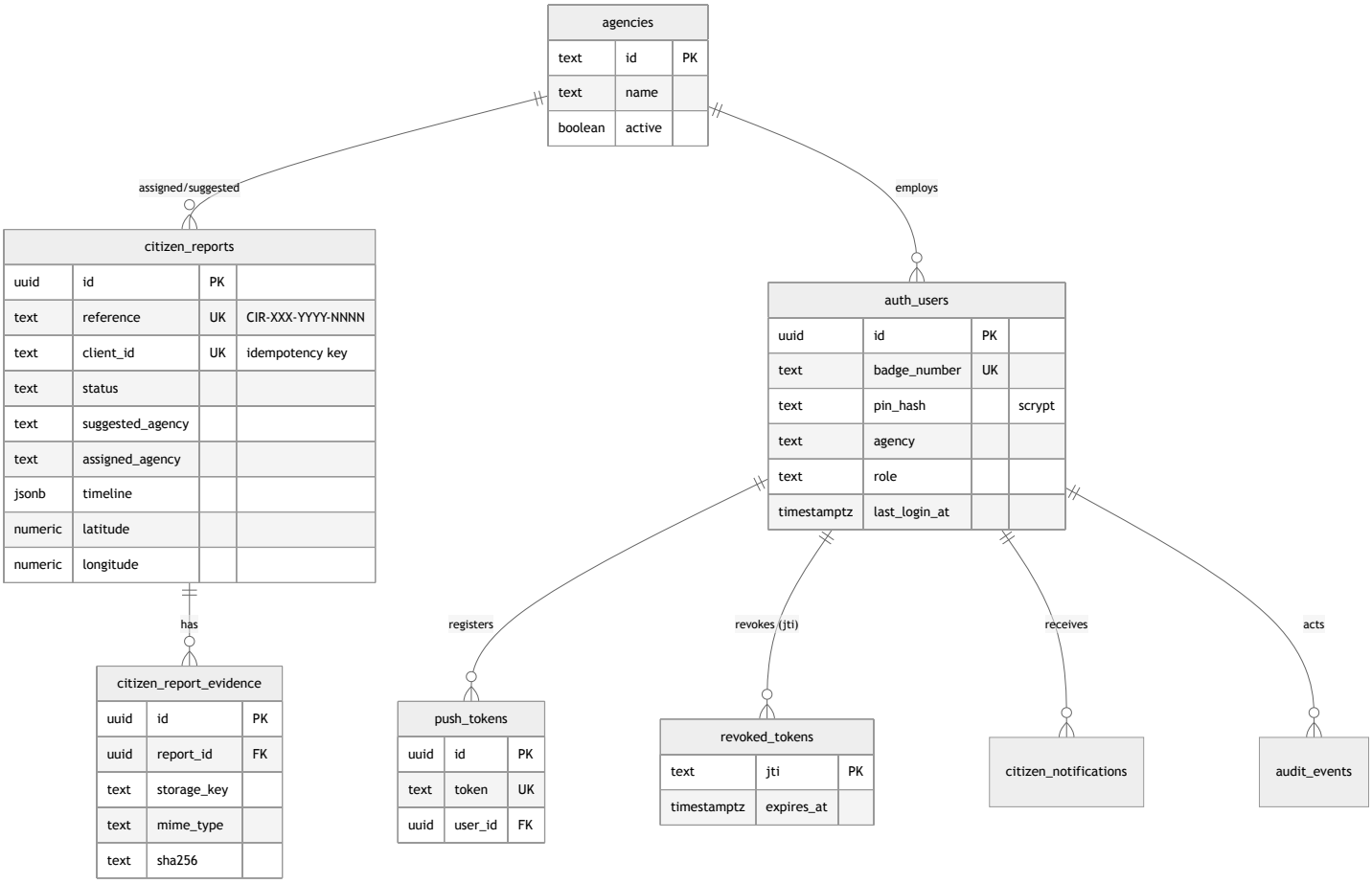
Evidence downloads (signed URLs)

Agency users never get raw file paths. They request `GET /api/reports/:id/evidence/:eid/download-url` (agency-scoped, audited), which mints a 15-minute HMAC-signed URL: `/api/evidence-files/:eid?exp=…&sig=…`. The download route verifies expiry and signature in constant time and streams the binary — browser-openable without a bearer token, but only with a valid signature. The binary backend is behind the `EvidenceBinaryStorage` interface (evidenceStorage.ts): local disk today (with path-traversal guards), S3/GCS is a drop-in implementation.

5. Data model

Two schema families coexist in lib/db/src/schema:

Flat mobile-mirror family — what the live mobile flows use. Plain-text agency ids so admin-created agencies (DSS, Fire Service, custom) need no tenant provisioning:



Tenant-scoped family (`tenants` , `agency_units` , `users` , `case_types` , `cases` , `evidence` , `referrals` , `duty_sessions` , `citizen_profiles` , `audit_logs`) — a richer UUID-FK model with per-tenant units and case types, used by the ops routes (duty sessions, referrals, case types). The long-term direction is to converge the flat family into this one; until then, both are maintained by the same forward-only migrations in `lib/db/migrations` (rollback = restore from backup).

Persistence mode is explicit: `DATABASE_URL` set → Postgres; unset → in-memory stores (dev/demo only, resets on restart). `GET /api/healthz` reports which mode is live and returns 503 if the database is unreachable.

6. Push notifications

Server side (pushSender.ts, pushDispatch.ts): every in-app notification is mirrored to Expo push (batched 100 per request, invalid tokens filtered, rejected receipts logged), targeted per-user or per-agency. OTP delivery rides the same path. Client side: tokens are registered on real devices only (Expo Go SDK 53+ cannot receive remote pushes), stored via `POST /api/push-tokens` , and notification taps deep-link to the route in the push payload — including cold starts.

7. Offline & degraded-mode matrix

Condition	Behavior
Phone offline	Reports queue locally (`pendingSync`), SyncBanner shows count, sync retries with stable `clientId` → no duplicates
API unreachable	Every repository falls back to AsyncStorage cache; app remains fully usable with local data (badged DEMO where applicable)
API up, no `DATABASE_URL`	Server runs on in-memory stores; healthz says `"db": "in-memory"` ; ops routes return 503
Database down	healthz 503 `db: "error"` ; mobile falls back locally
Expo Go (no push)	Registration returns `requiresDevelopmentBuild: true` ; in-app notification centre still works
No native maps (web/fallback)	`CitizenReportMap.tsx` renders the tappable list fallback; `.native.tsx` renders react-native-maps

8. Security architecture and known gaps

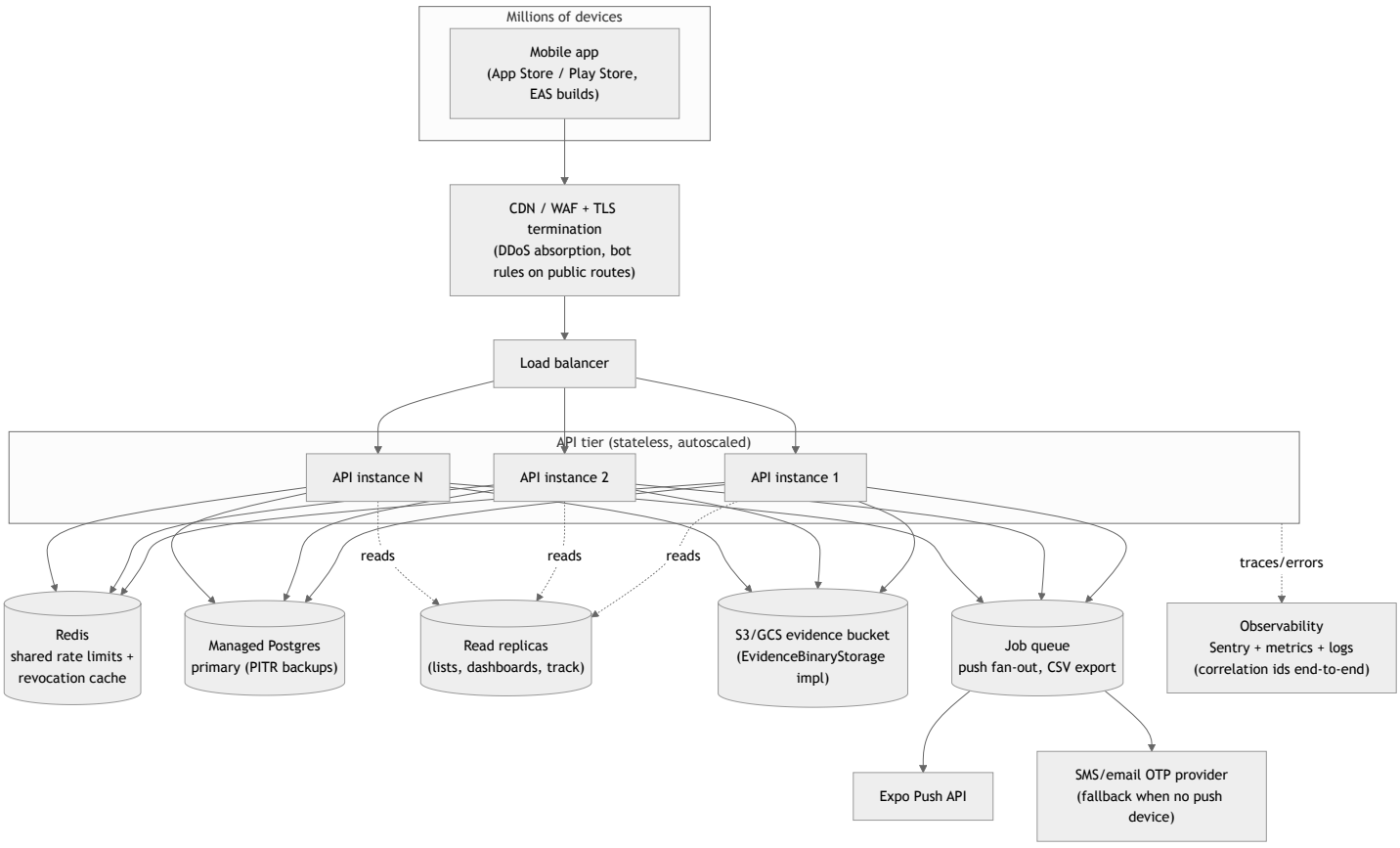
Defense layers in place: schema validation on every body (Zod from the OpenAPI contract), scrypt PIN hashing, HMAC tokens with rotation + DB-backed revocation, per-route rate limits, prototype-pollution rejection, strict security headers, MIME + size validation with server-side checksums on uploads, path-traversal guards, signed expiring download URLs, audit events on sensitive actions, OTP delivery only to the account's own devices, supply-chain cooldown on npm installs.

Gaps that must close before production (each is a roadmap item):

1. **Legacy `mvp.ts` router** (src/routes/mvp.ts) is still mounted under `/api` and authenticates with client-supplied `x-agency` / `x-user-id` / `x-user-role` headers — any caller can claim any role. It bypasses the entire HMAC token model. **Remove it or gate it behind `NODE_ENV !== "production"`.** This is the single highest-priority fix.
2. **CORS is wide open** (`cors()` with no origin allowlist). Fine for a native app, but combined with token-less public routes it invites cross-origin abuse of the API from any website.
3. **Assistant endpoint is public and unlimited** (`POST /api/assistant/chat`, no auth, no rate limit) — it proxies to paid AI providers, so it is a direct cost-amplification target.
4. **Mobile auth is locally authoritative**: the app's seeded user list (PIN `1234`) can log a user into the UI even when the API rejects them; the API session is established best-effort. Production must flip this — server verdict is the login verdict, demo seeding disabled outside demo builds.
5. **Rate limits and token revocation are per-process** — a second API instance would not see the first one's counters or revocations. Redis is required before horizontal scaling (see §9).
6. **Single `AUTH_SECRET`** signs both tokens and evidence URLs with no rotation story. Move to a secret manager with dual-secret (old+new) verification windows.
7. **Anonymous evidence attach**: evidence create/upload on a report is public (rate-limited, validated) so citizens can attach photos — acceptable, but the upload should also be bound to knowledge of the report's `clientId` or a submission token so third parties can't attach files to someone else's report.

9. Production topology — the target

This is the end state the ROADMAP.md works backwards from: the same two containers, deployed for millions of users.



Why this shape works for this codebase specifically:

- **The API tier is already stateless** — HMAC tokens need no session store, so horizontal scaling is only blocked by the two per-process caches (rate limits, revocation), both of which move to Redis behind their existing interfaces.

- **Evidence storage is already an interface** — `EvidenceBinaryStorage` gets an S3 implementation; signed-URL semantics stay identical (or delegate to S3 presigned URLs).
- **The hot public reads** (`track/:reference` , agency lists, dashboards) are read-mostly and shard naturally onto replicas.
- **Push fan-out is already fire-and-forget** — moving it from in-process to a queue changes the call site, not the semantics.
- **The contract chain means clients don't break** — old app versions keep working as long as `openapi.yaml` evolves additively; enforce that in CI with a spec-diff check.

10. File map (where to look)

Concern	Files
Server entry & middleware	`artifacts/api-server/src/{app.ts, index.ts}` , `src/middlewares/`
Auth & tokens	`artifacts/api-server/src/lib/{auth.ts, otpStore.ts, password.ts}` , `src/routes/auth.ts`
Reports & lifecycle	`src/routes/{citizen-reports.ts, reports.ts}` , `src/lib/citizenReportStore.ts`
Evidence	`src/lib/evidenceStorage.ts` , `src/routes/evidence.ts`
Push	`src/lib/{pushSender.ts, pushDispatch.ts}`
DB schema & migrations	`lib/db/src/schema/index.ts` , `lib/db/migrations/`
API contract & codegen	`lib/api-spec/{openapi.yaml, orval.config.ts}` , `lib/api-zod/` , `lib/api-client-react/`
Mobile data layer	`artifacts/mobile/services/` (apiClient, apiConfig, *Repository)
Mobile auth & RBAC	`artifacts/mobile/context/AuthContext.tsx` , `artifacts/mobile/lib/permissions.ts`
Offline sync	`artifacts/mobile/context/IncidentContext.tsx` , `components/SyncBanner.tsx`
Navigation	`artifacts/mobile/app/` (route groups per agency)
Tests	`artifacts/api-server/tests/` , `artifacts/mobile/tests/`

Roadmap — Working Backwards from Production

This plan starts from the end state and derives the exact steps to get there. The end state is defined first; every phase below it exists only because the end state requires it. Architecture details and diagrams are in ARCHITECTURE.md; the target topology is §9 there.

The end state (what "done" means)

The app is live in the App Store and Play Store under `ng.gov.securityapp`. Millions of citizens can submit and track reports without an account; tens of thousands of agency users across FRSC, Police, VIO, NSCDC, and admin-created agencies work their queues with push notifications. Concretely, production means:

- **E1. No trust in the client.** Every authorization decision is made server-side; the server's login verdict is the only login verdict; no demo credentials exist in production builds.
- **E2. No single-instance assumptions.** N stateless API instances behind a load balancer; rate limits and token revocation shared via Redis; evidence in S3/GCS; managed Postgres with PITR backups and read replicas.
- **E3. Abuse-resistant public surface.** WAF/CDN in front; every public endpoint rate-limited, validated, and idempotent; the assistant endpoint can't be used as a free AI proxy.
- **E4. Observable.** An on-call engineer can trace one citizen's failed report from the mobile release version to the API instance to the SQL statement via the correlation id, and alerts fire before users notice.
- **E5. Verified on real devices.** Push, offline sync, evidence upload, and login proven on physical iPhone and Android with production builds.
- **E6. Recoverable.** Backups restore-tested; secrets rotatable without logout storms; a bad migration can't reach production (CI applies every migration to a fresh database first).

Working backwards: E5 requires store builds (Phase 5), which require production infrastructure to point at (Phase 4), which is only safe to expose once the security gaps are closed (Phase 2) and observable (Phase 3), which is only trustworthy with CI guarding every merge (Phase 1 — **done in this pass**). Read the phases top to bottom as the execution order.

Phase 1 — Engineering safety net ✅ (this pass)

> Enables everything: no phase below is safe to execute without CI catching regressions.

- [x] GitHub Actions CI (.github/workflows/ci.yml): frozen-lockfile install, workspace typecheck, API + mobile test suites, diff hygiene, and a migration job against fresh Postgres 16.
- [ ] Add ESLint + `typescript-eslint` + `eslint-plugin-security` and wire into CI (commands in TOOLING.md).
- [ ] Add the OpenAPI drift check: CI runs `pnpm --filter @workspace/api-spec codegen` and fails if `git status --porcelain lib/` is non-empty.
- [ ] Enable GitHub secret scanning + push protection on the repository.

Phase 2 — Close the security gaps (blocks any public exposure)

> Derived from E1 and E3. Ordered by severity; items 1–3 are prerequisites for ever pointing a real domain at this API. Details in ARCHITECTURE.md §8.

1. **Remove the legacy `mvp.ts` router from production** (`artifacts/api-server/src/routes/mvp.ts`). It trusts client-supplied `x-agency` / `x-user-role` headers and bypasses token auth entirely. Either delete it (preferred — the flat model has superseded it) or mount it only when `NODE_ENV !== "production"`. Add a test asserting `/api/tenants` 404s in production mode.
2. **Lock down the assistant endpoint**: add `requireAuth` or an aggressive per-IP rate limit (it proxies paid Anthropic/Gemini calls), plus a max-tokens/day budget guard.
3. **CORS allowlist**: replace open `cors()` with an explicit origin list (native apps send no Origin; the allowlist exists for the web/admin surface).
4. **Make the server the login authority**: in API mode, a server rejection must fail the mobile login (today `AuthContext` logs in locally and attaches the API session best-effort). Gate demo seeding behind a build flag that is off in production profiles.
5. **Bind evidence attach to the submitter**: require the report's `clientId` (or a one-time submission token returned at create) on `POST /reports/:ref/evidence` so strangers can't attach files to others' reports.

6. **Secret rotation**: support `AUTH_SECRET` + `AUTH_SECRET_PREVIOUS` verification so the secret can rotate without invalidating every session at once.
7. Run `pnpm audit --prod` clean; add it to CI.

Exit criteria: external pentest-style pass of every route in ARCHITECTURE.md's route table finds no authorization bypass; all Phase 2 items have regression tests.

Phase 3 — Observability and load proof

> Derived from E4. Do this before real infrastructure so the first deploy is already measurable.

1. Sentry on both sides: `sentry-expo` in the mobile app (release-tagged), `@sentry/node` in the API, joined by the existing `X-Request-Id` correlation id.
2. Metrics endpoint (Prometheus or provider-native): request rate/latency/error by route, rate-limit rejections, push delivery failures, DB pool saturation.
3. Alerts: healthz failures, error-rate > budget, p95 latency > budget, push-receipt rejection spikes.
4. k6 load scripts for the public surface (`POST /citizen-reports` with unique clientIds, `GET /track/:reference`, evidence upload). Record the single-instance ceiling — this number sizes Phase 4's autoscaling.

Exit criteria: a induced failure (kill DB, saturate rate limit) pages with a trace that names the failing route and instance.

Phase 4 — Production infrastructure

> Derived from E2 and E6. Everything here slots into interfaces that already exist — no application rewrites.

1. **Managed Postgres** (RDS/Cloud SQL/Neon) with PITR; set `DATABASE_URL` via a secret manager; run `pnpm --filter @workspace/db run migrate` as a deploy step (CI already proves migrations apply cleanly).
2. **Redis**: implement the shared backend for `src/lib/rateLimit.ts` counters and the revocation check in `src/lib/auth.ts` (both are small interfaces today). This is the one code change that unlocks N > 1 instances.
3. **S3/GCS evidence storage**: implement `EvidenceBinaryStorage` (put / createReadStream / exists) against a private bucket; keep the HMAC signed-URL scheme or swap to native presigned URLs.
4. **Compute**: containerize the API (it builds to a single `dist/index.mjs`); deploy ≥2 instances behind a load balancer with `/api/healthz` as the health check; TLS + WAF/CDN in front with bot rules on the public routes.
5. **Secrets**: AUTH_SECRET, DATABASE_URL, ANTHROPIC/GEMINI keys, EXPO_ACCESS_TOKEN in the platform secret manager; nothing in env files on disk.
6. **SMS/email OTP provider** (e.g. Termii/Twilio for Nigerian numbers) as the OTP fallback for users without a registered push device — plugs into `deliverOtp` in `src/lib/pushDispatch.ts`.
7. Restore-test a backup into a scratch database; document the runbook.

Exit criteria: two API instances serve traffic simultaneously; killing one loses no requests; a token revoked on instance A is rejected by instance B; k6 run from Phase 3 passes against the real stack at target load.

Phase 5 — Mobile release

> Derived from E5. Requires accounts/devices (Apple Developer membership, Android keystore) — the one phase that cannot be done from this repo alone.

1. Apple Developer team + `eas credentials` for iOS signing and the Android keystore; confirm `ng.gov.securityapp` bundle id/package (already in `app.json`).
2. Point `production` profile env at the real API domain (`EXPO_PUBLIC_API_BASE_URL=https://api.<domain>/api` in `eas.json` — it currently holds a LAN IP for development).
3. `eas build --profile production` for both platforms; development-build QA pass on physical iPhone and Android: login, citizen report + track, evidence upload, push receipt + deep link, offline sync (airplane-mode test), agency reassignment flow.
4. Store listings, privacy policy (the app collects location + photos — both stores require disclosure), data-safety forms.
5. Staged rollout: TestFlight / Play internal track → closed pilot → production with phased rollout percentage.

Exit criteria: the ten flows in the README's test list pass on physical devices against production infrastructure.

Phase 6 — Pilot, then scale

> The end state says "millions"; you get there by proving one state/agency first.

1. Pilot with one agency command (e.g. one FRSC state command): seed real users via `/admin/users`, retire demo accounts, watch dashboards for a full duty cycle.
2. Tune from real numbers: rate-limit thresholds, autoscaling targets, push batch sizes, read-replica routing for `track/:reference` and dashboards.
3. Scale-out backlog (execute as load demands, not before):
   - Move push fan-out and CSV export to the job queue.
   - Read replicas for list/dashboard/track queries.
   - Converge the flat schema family into the tenant-scoped family (one data model, per ARCHITECTURE.md §5) once ops routes are the primary surface.
   - Partition `citizen_reports` by month if volume warrants (reference format already encodes year).
4. Quarterly: restore-test backups, rotate AUTH_SECRET (dual-secret window from Phase 2), re-run the k6 suite, dependency audit review.

One-page checklist (execution order)

#	Item	Phase	Status
1	CI: typecheck + tests + migration proof	1	✅ done
2	ESLint + security lint in CI	1	☐
3	OpenAPI drift check in CI	1	☐
4	Remove/gate `mvp.ts` header-auth router	2	☐ **highest priority**
5	Auth + rate limit on assistant endpoint	2	☐
6	CORS allowlist	2	☐
7	Server-authoritative mobile login; no demo seed in prod	2	☐
8	Evidence attach bound to submitter	2	☐
9	AUTH_SECRET rotation support	2	☐
10	Sentry both sides + metrics + alerts	3	☐
11	k6 load baseline	3	☐
12	Managed Postgres + PITR + deploy-time migrate	4	☐
13	Redis for rate limits + revocation	4	☐
14	S3/GCS `EvidenceBinaryStorage`	4	☐
15	Container + LB + WAF/TLS, ≥2 instances	4	☐
16	Secret manager for all secrets	4	☐
17	SMS/email OTP fallback provider	4	☐
18	Backup restore test + runbook	4	☐
19	iOS/Android credentials, prod API URL in eas.json	5	☐
20	Physical-device QA (10 README flows)	5	☐
21	Store listings + privacy disclosures + staged rollout	5	☐
22	Single-command pilot, then scale-out backlog	6	☐

Tooling

This document lists the tools the workspace uses today, the tools added as part of the architecture hardening pass, and the tools you should adopt before production. For the system design itself see ARCHITECTURE.md; for the ordered plan see ROADMAP.md.

In use today

Tool	Where	Purpose
pnpm workspaces	`pnpm-workspace.yaml`	Monorepo package management. `minimumReleaseAge: 1440` is a deliberate supply-chain defense — do not disable it.
TypeScript 5.9 (project references)	`tsconfig.base.json`, `pnpm run typecheck`	Whole-workspace type safety. Libs build with `tsc --build`; artifacts typecheck per package.
Express 5 + Zod	`artifacts/api-server`	HTTP API with runtime validation at every boundary.
Drizzle ORM + drizzle-kit	`lib/db`	Postgres schema, forward-only SQL migrations (`lib/db/migrations`).
Orval	`lib/api-spec/orval.config.ts`	Codegen from `openapi.yaml` → react-query client (`lib/api-client-react`) and Zod validators (`lib/api-zod`). The OpenAPI spec is the single source of truth for the API contract.
esbuild	`artifacts/api-server/build.mjs`	Server bundle.
Vitest + supertest	`artifacts/api-server/tests`, `artifacts/mobile/tests`	API tests (auth, RBAC, lifecycle, evidence, rate limits) and mobile repository tests. Hermetic: no database needed.
Expo / EAS	`artifacts/mobile/eas.json`	Mobile builds; profiles pinned in July 2026 build-out.
pino / pino-http	api-server	Structured logs with request correlation ids (`X-Request-Id`).
Prettier	root `devDependencies`	Formatting.

Added in this pass

Tool	Where	Purpose
GitHub Actions CI	`.github/workflows/ci.yml`	On every push/PR: frozen-lockfile install, workspace typecheck, API server tests, mobile tests, diff hygiene — plus a second job that applies all Drizzle migrations against a fresh Postgres 16 so a broken migration can never merge.

Adopt before production (in priority order)

1. **ESLint (flat config) + `typescript-eslint` + `eslint-plugin-security`** — typecheck catches type errors, not logic smells (floating promises, unhandled rejections, insecure patterns). Wire into the CI `check` job. `pnpm add -D -w eslint typescript-eslint eslint-plugin-security`
2. **Dependency audit in CI** — `pnpm audit --prod --audit-level high` as a CI step, plus GitHub Dependabot/Renovate with a cooldown matching `minimumReleaseAge`.
3. **Secret scanning** — enable GitHub secret scanning + push protection on the repo; `gitleaks` in CI as a backstop. The app signs auth tokens and evidence URLs with `AUTH_SECRET`; a leaked secret is a full-authentication bypass.
4. **Load testing** — `k6` scripts against the public endpoints (`POST /api/citizen-reports`, `GET /api/citizen-reports/track/:reference`) since these are unauthenticated and will absorb the worst abuse. Gate releases on p95 latency and error-rate budgets.
5. **Error tracking + APM** — Sentry (Expo has first-class support; pair with the server SDK so one incident links mobile release ↔ API trace via the correlation id).
6. **OpenAPI drift check** — CI step that runs Orval codegen and fails on a dirty diff, so `openapi.yaml`, the Zod validators, and the generated client can never diverge.
7. **Database backups + PITR** — managed Postgres (Neon/RDS/Cloud SQL) with point-in-time recovery; test restores quarterly. Migrations are forward-only, so backups are the rollback story.

8. **Redis** — shared rate-limit counters and token-revocation cache once the API runs on more than one instance (both are per-process `Map`s today; see ARCHITECTURE.md §9).

Everyday commands

```
nvm use 22                                  # pnpm needs Node 22+
pnpm install
pnpm run typecheck                          # whole workspace
pnpm --filter @workspace/api-server test    # API test suite
pnpm --filter @workspace/mobile test        # mobile test suite
pnpm --filter @workspace/api-spec codegen   # regenerate client + zod from openapi.yaml
pnpm --filter @workspace/db run generate    # new migration from schema change
pnpm --filter @workspace/db run migrate     # apply migrations
pnpm --filter @workspace/mobile run dev     # Expo dev server
pnpm --filter @workspace/api-server run dev # API on :8081
```